

Pointers

- **Pointers** are one of the core components of the C programming language. A pointer can be used to store the **memory address** of other variables, functions, or even other pointers.
- The use of pointers allows low-level memory access, dynamic memory allocation, and many other functionality in C.
- A pointer is defined as a derived data type that can store the address of other C variables or a memory location. We can access and manipulate the data stored in that memory location using pointers.
- As the pointers store the memory addresses, their size is independent of the type of data they are pointing to.

→ Syntax

The syntax of pointers is similar to the variable declaration in C, but we use the **(*) dereferencing operator** in the pointer declaration.

```
datatype * ptr;
```

Where,

- **ptr** is the name of the pointer.
- **datatype** is the type of data it is pointing to.

The above syntax is used to define a pointer to a variable. We can also define pointers to functions, structures, etc.

How to Use Pointers?

The use of pointers can be divided into three steps:

- Pointer Declaration
- Pointer Initialization
- Dereferencing

➤ Pointer Declaration

- In pointer declaration, we only declare the pointer but do not initialize it. To declare a pointer, we use the (*) **dereference operator** before its name.
- The pointer declared here will point to some random memory address as it is not initialized. Such pointers are called wild pointers.

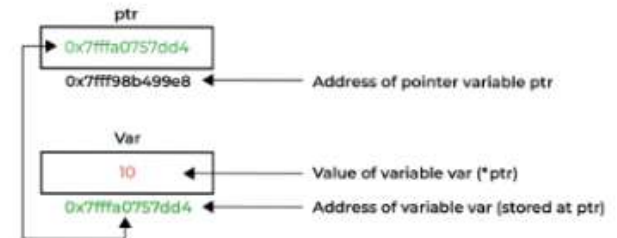
➤ Pointer Initialization

- Pointer initialization is the process where we assign some initial value to the pointer variable.
- We generally use the (&) **addressof operator** to get the memory address of a variable and then store it in the pointer variable.
- We can also declare and initialize the pointer in a single step. This method is called **pointer definition** as the pointer is declared and initialized at the same time.

Note: It is recommended that the pointers should always be initialized to some value before starting using it. Otherwise, it may lead to number of errors.

➤ Dereferencing

- Dereferencing a pointer is the process of accessing the value stored in the memory address specified in the pointer.
- We use the same (*) **dereferencing operator** that we used in the pointer declaration.



Example:

```
#include <stdio.h>

int main() {

    int num = 10; // Declare an integer variable

    int *ptr;    // Declare a pointer to an integer

    ptr = &num;  // Assign the address of 'num' to the pointer

    printf("Value of num: %d\n", num);

    printf("Address of num: %p\n", &num);

    printf("Value of ptr: %p\n", ptr);

    printf("Value pointed by ptr: %d\n", *ptr);

    return 0;

}
```

Output:

```
Value of num: 10
Address of num: 0x7ffd2a8c234c
Value of ptr: 0x7ffd2a8c234c
Value pointed by ptr: 10
```

Types of Pointers

Pointers can be classified into many different types based on the parameter on which we are defining their types.

If we consider the type of variable stored in the memory location pointed by the pointer, then the pointers can be classified into the following types:

1. Integer Pointers

- As the name suggests, these are the pointers that point to the integer values.
- These pointers are pronounced as **Pointer to Integer**.
- Similarly, a pointer can point to any primitive data type. It can point also point to derived data types such as arrays and user-defined data types such as structures.

2. Array Pointer

- Pointers and Array are closely related to each other. Even the array name is the pointer to its first element. They are also known as Pointer to Arrays.

3. Structure Pointer

- The pointer pointing to the structure type is called Structure Pointer or Pointer to Structure.
- It can be declared in the same way as we declare the other primitive data types.
- In C, structure pointers are used in data structures such as linked lists, trees, etc.

4. Function Pointers

- Function pointers point to the functions. They are different from the rest of the pointers in the sense that instead of pointing to the data, they point to the code.

- Let's consider a function prototype – **int func (int, char)**

Note: The syntax of the function pointers changes according to the function prototype.

5. Double Pointers

- In C language, we can define a pointer that stores the memory address of another pointer. Such pointers are called double-pointers or pointers-to-pointer.

6. NULL Pointer

- The Null Pointers are those pointers that do not point to any memory location. They can be created by assigning a NULL value to the pointer.
- A pointer of any type can be assigned the NULL value.
- It is said to be good practice to assign NULL to the pointers currently not in use.

7. Void Pointer

- The Void pointers in C are the pointers of type void. It means that they do not have any associated data type.
- They are also called **generic pointers** as they can point to any type and can be typecasted to any type.

8. Wild Pointers

- The Wild Pointers are pointers that have not been initialized with something yet.
- These types of C-pointers can cause problems in our programs and can eventually cause them to crash.

9. Constant Pointers

- In constant pointers, the memory address stored inside the pointer is constant and cannot be modified once it is defined.
- It will always point to the same memory address.

10. Pointer to Constant

- The pointers pointing to a constant value that cannot be modified are called pointers to a constant.
- Here we can only access the data pointed by the pointer, but cannot modify it. Although, we can change the address stored in the pointer to constant.

Pointer Arithmetic

The Pointer Arithmetic refers to the legal or valid arithmetic operations that can be performed on a pointer.

It is slightly different from the ones that we generally use for mathematical calculations as only a limited set of operations can be performed on pointers.

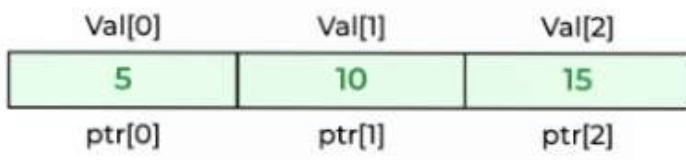
These operations include:

- Increment in a Pointer
- Decrement in a Pointer
- Addition of integer to a pointer
- Subtraction of integer to a pointer
- Subtracting two pointers of the same type
- Comparison of pointers of the same type.
- Assignment of pointers of the same type.

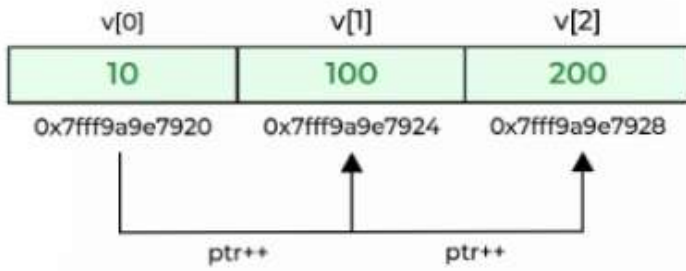
C Pointers and Arrays

- In C programming language, pointers and arrays are closely related. An array name acts like a pointer constant.
- The value of this pointer constant is the address of the first element. For example, if we have an array named val then **val** and **&val[0]** can be used interchangeably.
- If we assign this value to a non-constant pointer of the same type, then we can access the elements of the array using this pointer.

Example 1: Accessing Array Elements using Pointer with Array Subscript



Example 2: Accessing Array Elements using Pointer Arithmetic



➔ **Advantages of Pointers**

- Pointers are used for dynamic memory allocation and deallocation.
- An Array or a structure can be accessed efficiently with pointers
- Pointers are useful for accessing memory locations.
- Pointers are used to form complex data structures such as linked lists, graphs, trees, etc.
- Pointers reduce the length of the program and its execution time as well.

➔ **Disadvantages of Pointers**

- Memory corruption can occur if an incorrect value is provided to pointers.
- Pointers are a little bit complex to understand.
- Pointers are majorly responsible for memory leaks in C.
- Pointers are comparatively slower than variables in C.
- Uninitialized pointers might cause a segmentation fault.